Bijux Pollenomics: implementation and verification

Developer briefing

Bijux Pollenomics

30 September 2026

Source-bound research briefing

Distribution responsibilities

Distribution	Responsibility
`bijux-pollenomics`	Canonical collection, evidence, review and publication runtime.
`pollenomics`	Short-name compatibility facade.
`bijux-pollenomics-lra`	Optional REVEALS/LOVE engine and extension entry point.
`bijux-pollenomics-dev`	Repository checks, source consumers and report verification.

A local built-wheel extra install and R-free run passed; the engine is not yet published on PyPI.

Evidence: `reports/architecture/lra-distribution.md`; `packages/bijux-pollenomics/pyproject.toml`

Why the numerical backend remains Python

- A private C++17 forward batch matched the tested arithmetic but missed the prospective 2× validated-call latency gate at all three declared shapes.
- One larger synthetic worker used about 31 MB less peak memory with C++, but its median call was slower.
- We kept NumPy in production. A future extension would stay inside the LRA wheel and require source-R parity, buffer and draw contracts, a wheel matrix and a measured complete-workflow benefit.

The measured forward spike does not decide the performance of every LRA kernel.

Evidence: `reports/architecture/native-backend.md`; `docs/report/numerical-performance/native-forward/report.json`

Follow a claim through the system

Source member → custody receipt → normalized record → admission decision

→ sealed prediction or governed calculation → score → report/publication

- Physical site, chronology, taxon concept, denominator and permissions are separate gates.
- Result manifests pin files and producer identity; refusal rows remain part of the product.
- Public text links a claim to its governed table and scope.

Evidence: `reports/README.md`; `reports/science/english-lra-pollen-prediction.md`

Three numerical checks with different reach

1. Source-R calculation checks compare selected native algorithms and fixtures.
2. Author-example correspondence compares Python with independent author R over 14,074 supported mean/SE values.
3. Archive correspondence checks the exact shared 100–350 BP window and records genuine product gaps.

A passing implementation check cannot authenticate historical source revisions or ecological truth.

Evidence: `docs/report/method-certification/original-r-reference/README.md`; `reports/science/publication-correspondence.md`

Prediction contract: freeze before scoring

- Group by physical site or country before fitting; retain source identities and eligible support.
- Prepare predictors separately from target outcomes; publish prediction hash first.
- Score all eligible rows, record refused bins and report the same support for paired methods.
- Keep observed-pollen, LRA forward-pollen and vegetation targets in different result contracts.

Evidence: `reports/science/denmark-finland-norway-author-pollen.md`; `reports/science/english-lra-pollen-prediction.md`

Reports and build ownership

- `reports/` is the human-facing claim index and open-question register.
- `docs/report/` currently owns many governed numeric outputs and their producers; migrate by domain with all consumers.
- `docs/public/` carries curated, rights-safe explanations for the website.
- `presentation/` builds from locked report identities and refuses stale source changes.

Tracked report publication preserves governed files; independent full-tree rebuilding remains open.

Evidence: `reports/architecture/ownership.md`; `presentation/README.md`

Developer verification route

1. Run the selected producer with a declared input root into `artifacts/`.
2. Replay the specific report verifier and compare manifest identities.
3. Run affected tests, type/style checks and strict website build for changed pages.
4. Build all decks; a stale source lock must stop publication until reviewed.

Source rights and missing ecological inputs are scientific gates, not test failures to suppress.

Evidence: `docs/report/pollen-reconstruction/verify_report.py`; `presentation/build.py`